SOFTWARE REQUIREMENTS SPECIFICATION

# StaffDesk

## Phase 2 — Organisational Structure & Task Management

Change Request CR-2026-014 — Backend Training Program
Client: Crystal Mind — Business Management Solutions

| | |
|---|---|
| **Prepared by** | Mazen Adeeb, Engineering Mentor |
| **Prepared for** | Ahmed Ghazaly, Backend Developer |
| **Supersedes** | Nothing. Extends StaffDesk SRS v1.1 (Phase 1). |
| **Document Version** | 2.0 |
| **Date Issued** | 1 September 2026 |
| **Status** | Approved for Development |

# Document Control

| Version | Date | Author | Description |
|---|---|---|---|
| 1.0 | 26 Jul 2026 | Mazen Adeeb | StaffDesk Phase 1 — directory API. |
| 1.1 | 31 Aug 2026 | Mazen Adeeb | Phase 1 reissued to Ahmed Ghazaly; web UI added. |
| 2.0 | 1 Sep 2026 | Mazen Adeeb | **This document.** Phase 2: employee seniority and reporting lines, and a full task management system layered on the authenticated StaffDesk service. |

## Approval

| Role | Name | Signature / Approval |
|---|---|---|
| Engineering Mentor / Requester | Mazen Adeeb | Approved |
| Developer | Ahmed Ghazaly | Pending acknowledgment |

# Table of Contents

# 1. Introduction

## 1.1 Purpose

This document specifies **Phase 2** of StaffDesk. Phase 1 delivered a departments-and-employees directory; you have since added authentication and role-based permissions. Phase 2 turns that directory into a working internal tool by adding two things the business has asked for: a real **organisational structure** (seniority levels, job titles, reporting lines) and a complete **task management system** on top of it.

The brief is written the way a specification arrives on a real team: as a set of numbered, testable requirements with the edge cases named, rather than a paragraph of intent. It is deliberately longer and harder than Phase 1. Read all of it before writing any code.

## 1.2 Scope

In scope: seniority levels, reporting lines, department managers, and the full task lifecycle — creation, assignment, prioritisation, status transitions, comments, watchers, checklists, tags, dependencies, due dates, activity history, notifications, and the query surface needed to drive a task list and a board. Out of scope items are enumerated in Section 17, and are excluded on purpose.

## 1.3 Intended Audience

The assigned developer (Ahmed Ghazaly) and the reviewing mentor (Mazen Adeeb). It assumes the material of Training Days 1–14 and does not re-explain it.

## 1.4 Curriculum References

| Day | Training Document | Where it is exercised |
|---|---|---|
| 8 | CRUD APIs End-to-End | Full update/delete surface on tasks and levels (Section 12) |
| 9 | Authentication & Authorization | Permission matrix and 401/403 discipline (Section 8) |
| 10 | Testing Fundamentals | Automated tests are now a deliverable (NFR-11) |
| 11 | Validation & Error Handling | Error catalogue and centralized handler (Section 13) |
| 12 | API Design & Documentation | OpenAPI spec covering the whole surface (NFR-9) |
| 13 | Debugging & Logging | Structured request and audit logging (NFR-12) |
| 14 | Reading a Real Codebase | Extending your own Phase 1 code without rewriting it |

## 1.5 How to Read This Document

Requirements are tagged **MUST** (required for acceptance), **SHOULD** (expected; its absence must be justified in writing), or **MAY** (optional, no effect on acceptance). Requirement IDs are stable and are the vocabulary for review comments — expect pull request feedback of the form “TR-18 is not satisfied here”.

Anything this document does not pin down is a decision delegated to you. Delegated decisions must be recorded in the repository’s `docs/decisions.md`, one short paragraph each: what you chose, what you rejected, and why. That file is part of the deliverable.

# 2. Current System Baseline & Assumptions

This specification is written against the system as delivered at the end of Phase 1 plus your authentication work. Because that work was yours, the following assumptions may not match what you built exactly. **Reconcile them before starting.** Where an assumption is wrong, adapt the requirement to your actual codebase and note the deviation in `docs/decisions.md` — do not rebuild working code to match a guess in this document.

| ID | Assumed current state | If this is wrong… |
|---|---|---|
| **A-1** | Departments and Employees exist with the Phase 1 fields, in PostgreSQL via Prisma. | Adapt field names; the relationships must still hold. |
| **A-2** | A User (or Account) record exists carrying credentials, and is linked one-to-one to an Employee. | If identity and employee are one table, treat every “user” below as that table. |
| **A-3** | Login issues a JWT; middleware rejects unauthenticated requests with 401. | Phase 2 assumes an authenticated principal on every request. This is a prerequisite. |
| **A-4** | A coarse role exists, roughly ADMIN vs ordinary employee, checked in middleware. | Section 8 defines the roles Phase 2 needs; extend rather than replace. |
| **A-5** | Passwords are bcrypt-hashed and never returned by any endpoint. | Fix this first, before any Phase 2 work. |
| **A-6** | Search and pagination exist on the employee list. | The task list reuses the same conventions; align them. |

### 2.1 Compatibility Requirement

Phase 2 is an **extension**, not a rewrite. Every Phase 1 endpoint must keep working, with the same paths and the same response shapes, except where a requirement here explicitly changes one. Where a response gains a field, that is acceptable; where a field is removed or renamed, it is a breaking change and must be called out in the pull request description.

> **A NOTE ON SIZE**
>
> This document asks for considerably more than Phase 1 did. That is intentional: the point is to practise reading a large specification, decomposing it into deliverable slices, and reporting progress against it — not to write all of it in one sitting. Section 20 breaks the work into four milestones. If a milestone slips, say so at the time; a slipping estimate reported early is a normal engineering event, a silent one is not.

## 3. Business Context & Objectives

Crystal Mind’s internal teams currently track work in a mix of spreadsheets and chat messages. Two problems recur: nobody can answer “what is this person working on right now?” without asking them, and nobody can answer “who is allowed to give this person work?” at all. Phase 1’s directory answered *who exists*. Phase 2 must answer *who reports to whom* and *who is doing what*.

### 3.1 Objectives

- **O-1 — Make the hierarchy explicit.** Every employee has a seniority level and, where applicable, a manager. Every department has one manager.
- **O-2 — Make work trackable.** Any unit of work is a task with an owner, a priority, a state, and a history that explains how it reached that state.

- **O-3 — Make authority enforceable.** Who may assign, reassign, close, or delete a task is derived from the hierarchy and enforced by the API, not by convention.
- **O-4 — Make the data answerable.** A manager can retrieve their team's open work, overdue items, and per-status counts in a single request each.

## 3.2 Success Measures

| # | Measure | How it will be checked |
|---|---|---|
| **SM-1** | Every task action is attributable. | For any task, the activity endpoint reconstructs who changed what, and when, since creation. |
| **SM-2** | No privilege escalation via the API. | Attempting each forbidden action in Section 8 returns 403, verified by automated tests. |
| **SM-3** | A manager's dashboard needs ≤ 3 requests. | Team task list, per-status summary, and overdue list are each one call. |
| **SM-4** | The task list stays responsive at 5,000 tasks. | Seeded volume test; list endpoint returns within a documented budget (NFR-6). |

# 4. Part A — Organisational Structure & Employee Titles

## 4.1 Three Concepts That Must Not Be Confused

The single most common mistake in this feature is collapsing three distinct ideas into one column. They vary independently and are used for different purposes:

| Concept | Example values | What it is for |
|---|---|---|
| **Job title** | “Backend Developer”, “QA Analyst”, “Office Manager” | Free-text label describing what the person does. Display only. Never drives logic. |
| **Seniority level** | Intern, Junior, Mid, Senior, Lead, Manager, Director | Ordered, reference-data-driven rank. Drives assignment rules (Section 7). |
| **System role** | ADMIN, MANAGER, MEMBER | Permission bucket used by authorization middleware (Section 8). |

A Senior Developer and a Senior QA Analyst share a level and differ in title. A Manager-level employee who is not managing anyone this quarter still holds the MEMBER role. Keep the three columns separate.

## 4.2 Requirements

| ID | Requirement | Priority |
|---|---|---|
| **OR-1** | A **SeniorityLevel** shall be a database-backed reference entity, not a hard-coded enum in application code, so that levels can be added without a deployment. | **MUST** |
| **OR-2** | Each seniority level shall carry a unique name, a unique integer *rank* establishing order (higher rank = more senior), an optional description, and an active flag. | **MUST** |
| **OR-3** | The system shall be seeded with at least: Intern (10), Junior (20), Mid (30), Senior (40), Lead (50), Manager (60), Director (70). Ranks are spaced by ten so levels can be inserted between them later. | **MUST** |
| **OR-4** | Employee shall gain a required `levelId` foreign key to SeniorityLevel, and shall keep `jobTitle` as free text. | **MUST** |
| **OR-5** | Employee shall gain an optional self-referencing `managerId` identifying that employee’s direct manager. | **MUST** |
| **OR-6** | The reporting line shall be acyclic. An employee may not manage themselves, and no chain of managers may loop back to its start. Attempting to create a cycle shall be rejected with 422. | **MUST** |
| **OR-7** | Department shall gain an optional `managerId` identifying the department’s manager. That employee must belong to the department they manage. | **MUST** |
| **OR-8** | Employee shall gain an `isActive` boolean, defaulting to true. Deactivated employees remain readable and remain attached to historical tasks, but shall not be assignable and shall not be able to log in. | **MUST** |
| **OR-9** | Full CRUD shall be exposed for seniority levels, restricted to ADMIN. | **MUST** |
| **OR-10** | Deleting a seniority level that is referenced by any employee shall be refused with 409. Deactivating it instead shall be permitted, and shall block its use on new or updated employees while leaving existing ones untouched. | **MUST** |
| **OR-11** | An endpoint shall return an employee’s direct reports; a second shall return the full reporting chain upward to the top of the hierarchy. | **SHOULD** |
| **OR-12** | Employee responses shall include the level’s name and rank, the manager’s id and name, and the department’s name — resolved server-side, not left to the client to join. | **MUST** |

| ID | Requirement | Priority |
|---|---|---|
| **OR-13** | Changing an employee's level, manager, department, or active flag shall be recorded in the activity log (Section 9.3) with the previous and new values. | **SHOULD** |
| **OR-14** | Employee list filtering shall support `levelId`, `departmentId`, `managerId`, and `isActive`, combinable with the existing search and pagination. | **MUST** |

## 4.3 Data Model — Part A

### *SeniorityLevel*

| Field | Type | Specification |
|---|---|---|
| **id** | `Int` | Primary key, auto-generated. |
| **name** | `String` | Required, unique, e.g. "Senior". Max 40 characters. |
| **rank** | `Int` | Required, unique. Higher is more senior. Used for ordering and for assignment rules. |
| **description** | `String?` | Optional free text, max 200 characters. |
| **isActive** | `Boolean` | Defaults to true. Inactive levels cannot be applied to an employee. |
| **employees** | `Employee[]` | One-to-many back-relation. |

### *Employee — added fields*

| Field | Type | Specification |
|---|---|---|
| **levelId** | `Int` | Required. Foreign key to SeniorityLevel. Restrict on delete. |
| **managerId** | `Int?` | Optional self-relation to Employee. Null for the top of a chain. Must not create a cycle (OR-6). |
| **isActive** | `Boolean` | Defaults to true. |
| **directReports** | `Employee[]` | Back-relation of managerId. |

### *Department — added field*

| Field | Type | Specification |
|---|---|---|
| **managerId** | `Int?` | Optional foreign key to Employee. The employee must have that department as their own departmentId. |

> **MIGRATION CARE**
>
> `levelId` is required, but the employees table already has rows. A single migration adding a NOT NULL column with no default will fail. Plan the migration in steps — add nullable, backfill every existing row to a sensible default level, then tighten to NOT NULL — and commit each step. Doing this correctly on data that already exists is the actual exercise here.

# 5. Part B — Task Management: Core Model

## 5.1 Design Reference

The model below is a deliberately reduced version of what commercial task systems expose. The reductions are as important as the inclusions — build what is listed, not what you have seen elsewhere:

| Concept in commercial tools | StaffDesk Phase 2 decision |
|---|---|
| Custom fields per project / per list | **Excluded.** A fixed schema only. Custom fields are a Phase 3 conversation. |
| Multiple workspaces, spaces, folders, lists | **Reduced** to the existing Department. A task belongs to exactly one department. |
| Per-list configurable status sets | **Reduced** to one fixed workflow for the whole system (Section 6). |
| Multiple assignees per task | **Reduced** to exactly one assignee, plus watchers for everyone else (Section 9.2). |
| Automations, triggers, recurring tasks | **Excluded** from acceptance; one narrow case is a stretch goal (Section 18). |
| Sprints, story points, burndown | **Excluded.** Due dates and estimates only. |
| Nested subtasks to arbitrary depth | **Reduced** to one level: a task may have subtasks; a subtask may not. |
| Rich text, attachments, embedded files | **Reduced** to plain text descriptions and comments, plus external URL references. No file uploads. |

## 5.2 Core Task Requirements

| ID | Requirement | Priority |
|---|---|---|
| **TR-1** | The system shall define a **Task** resource owned by exactly one department and belonging to exactly one creator. | **MUST** |
| **TR-2** | Every task shall carry a human-readable reference key, unique across the system, of the form `TSK-1042`. The key shall be generated by the server, shall never be reused, and shall never change — including after the task is deleted or moved between departments. | **MUST** |
| **TR-3** | A task shall have a required title of 3–140 characters and an optional plain-text description of up to 5,000 characters. | **MUST** |
| **TR-4** | A task shall have a status drawn from the fixed set in Section 6, defaulting to `OPEN` on creation. | **MUST** |
| **TR-5** | A task shall have a priority drawn from `URGENT, HIGH, NORMAL, LOW`, defaulting to `NORMAL`. | **MUST** |
| **TR-6** | A task shall have an optional assignee (an active Employee). A task with no assignee is *unassigned* and shall be represented by a null assignee, never by a sentinel employee record. | **MUST** |
| **TR-7** | A task shall record its creator (the authenticated employee who created it). The creator shall be immutable. | **MUST** |
| **TR-8** | A task shall have an optional due date. Due dates shall be stored as timestamps in UTC and returned in ISO-8601 with an explicit offset. | **MUST** |

| ID | Requirement | Priority |
|---|---|---|
| **TR-9** | A task shall record `createdAt`, `updatedAt`, and `completedAt`. `completedAt` shall be set automatically when the task first enters `DONE` and cleared if it is reopened. | **MUST** |
| **TR-10** | A task shall carry an optional integer `estimateMinutes` and an accumulated `loggedMinutes` (Section 10.3). | **SHOULD** |
| **TR-11** | Tasks shall be soft-deleted: a deleted task is excluded from all list and detail endpoints for ordinary callers, but its row, its history, and its key are retained. | **MUST** |
| **TR-12** | A task shall support an `isArchived` flag, distinct from deletion, excluding it from default listings while keeping it retrievable by explicit filter. | **SHOULD** |
| **TR-13** | Creating a task shall require a title and a departmentId. All other fields shall be optional and shall take documented defaults. | **MUST** |
| **TR-14** | The task detail response shall embed the assignee, creator, and department as small nested objects containing at minimum id and name — never bare foreign keys alone. | **MUST** |
| **TR-15** | A task may be moved to a different department. Moving a task whose assignee is not a member of the destination department shall unassign it and record both changes in the activity log. | **SHOULD** |

## 5.3 Data Model — Task

| Field | Type | Specification |
|---|---|---|
| **id** | `Int` | Primary key, auto-generated. |
| **key** | `String` | Unique, server-generated, immutable. Format TSK-. |
| **title** | `String` | Required, 3–140 characters, trimmed. |
| **description** | `String?` | Optional, up to 5,000 characters, plain text. |
| **status** | `TaskStatus` | Enum, defaults to OPEN. See Section 6. |
| **priority** | `TaskPriority` | Enum, defaults to NORMAL. |
| **departmentId** | `Int` | Required. Foreign key to Department. |
| **assigneeId** | `Int?` | Optional. Foreign key to Employee. Must be active and permitted (Section 7). |
| **createdById** | `Int` | Required, immutable. Foreign key to Employee. |
| **parentTaskId** | `Int?` | Optional. Foreign key to Task, one level only (TR-33). |
| **dueAt** | `DateTime?` | Optional, stored UTC. |
| **estimateMinutes** | `Int?` | Optional, positive. |
| **loggedMinutes** | `Int` | Defaults to 0. Derived from time entries; never written directly by a client. |
| **isArchived** | `Boolean` | Defaults to false. |
| **deletedAt** | `DateTime?` | Null unless soft-deleted. |
| **createdAt** | `DateTime` | Set on insert. |
| **updatedAt** | `DateTime` | Maintained on every write. |
| **completedAt** | `DateTime?` | Set on first entry to DONE; cleared on reopen. |

### 5.4 Supporting Entities

| Entity | Purpose | Key fields |
|---|---|---|
| **TaskComment** | Threaded discussion on a task. | id, taskId, authorId, body, createdAt, updatedAt, editedAt?, deletedAt? |
| **TaskActivity** | Append-only audit of every change. | id, taskId, actorId, action, field?, oldValue?, newValue?, createdAt |
| **TaskWatcher** | Employees following a task. | taskId, employeeId, createdAt — composite unique |
| **TaskChecklistItem** | Lightweight to-dos inside one task. | id, taskId, label, isDone, position, completedAt?, completedById? |
| **Tag** | Reusable label, shared system-wide. | id, name (unique, lowercased), colour? |
| **TaskTag** | Many-to-many join between Task and Tag. | taskId, tagId — composite primary key |
| **TaskDependency** | “Blocked by” relationship between two tasks. | id, blockedTaskId, blockingTaskId, createdAt — unique pair |
| **TaskTimeEntry** | A logged period of work. | id, taskId, employeeId, minutes, note?, workedOn, createdAt |
| **Notification** | In-app alert for one recipient. | id, recipientId, taskId?, type, message, isRead, createdAt |

# 6. Task Status Workflow

### 6.1 States

| Status | Meaning | Category |
|---|---|---|
| **OPEN** | Accepted, ready to be worked, not started. The default on creation. | Active |
| **IN_PROGRESS** | Someone is actively working on it. Requires an assignee. | Active |
| **BLOCKED** | Cannot proceed. Requires a reason (Section 6.3). | Active |
| **IN_REVIEW** | Work is complete and awaiting checking by someone other than the assignee. | Active |
| **DONE** | Accepted as complete. Sets completedAt. | Terminal |
| **CANCELLED** | Abandoned; will not be done. Distinct from deleted. | Terminal |

### 6.2 Transition Matrix

A transition not listed below shall be rejected with **409 Conflict** and error code `TASK_INVALID_TRANSITION`. Rejection must name both the current and the attempted status.

| From | To | Who may perform it | Conditions |
|---|---|---|---|
| **OPEN** | **IN_PROGRESS** | Assignee, creator, department manager, ADMIN | Task must have an assignee. If the actor is the assignee-to-be and the task is unassigned, they may claim it in the same call. |
| **OPEN** | **CANCELLED** | Creator, department manager, ADMIN | Reason required. |
| **IN_PROGRESS** | **IN_REVIEW** | Assignee, department manager, ADMIN | — |

| From | To | Who may perform it | Conditions |
|---|---|---|---|
| **IN_PROGRESS** | **BLOCKED** | Assignee, department manager, ADMIN | Reason required. |
| **IN_PROGRESS** | **OPEN** | Assignee, department manager, ADMIN | Used when work is put down. Does not clear the assignee. |
| **IN_PROGRESS** | **DONE** | Assignee, department manager, ADMIN | Permitted only when the task has no open checklist items and no unresolved blockers. |
| **BLOCKED** | **IN_PROGRESS** | Assignee, department manager, ADMIN | Clears the block reason. |
| **BLOCKED** | **CANCELLED** | Creator, department manager, ADMIN | Reason required. |
| **IN_REVIEW** | **DONE** | Anyone permitted to review — *not* the assignee (see 6.4) | — |
| **IN_REVIEW** | **IN_PROGRESS** | Reviewer, department manager, ADMIN | Rejection back to the assignee. Comment required. |
| **DONE** | **OPEN** | Creator, department manager, ADMIN | Reopening. Clears completedAt. Reason required. |
| **CANCELLED** | **OPEN** | Creator, department manager, ADMIN | Reason required. |

### 6.3 Transition Rules

| ID | Requirement | Priority |
|---|---|---|
| **TR-16** | Status shall be changed only through a dedicated transition endpoint (`PATCH /v1/tasks/:id/status`), never by a general field update. A general update that includes a status field shall be rejected with 400. | **MUST** |
| **TR-17** | Transitions marked “Reason required” shall require a non-empty `reason` of at least 5 characters in the request body; the reason shall be stored on the resulting activity record. | **MUST** |
| **TR-18** | A transition into `IN_PROGRESS` on an unassigned task shall be rejected with 422 unless the same request assigns it, and the actor is permitted to make that assignment. | **MUST** |
| **TR-19** | A transition into `DONE` shall be refused with 422 while the task has any incomplete blocking dependency (Section 10.2). | **MUST** |
| **TR-20** | Every transition shall write exactly one activity record capturing actor, from-status, to-status, reason, and timestamp. | **MUST** |
| **TR-21** | Transitions shall be applied inside a database transaction together with their activity record and any resulting notifications, so a failure leaves no partial state. | **MUST** |

### 6.4 The Review Rule

An assignee may move their own task to `IN_REVIEW` but may not move it from `IN_REVIEW` to `DONE`. Someone else must accept it. The single exception is a task whose creator and assignee are the same person and whose department has no other active member — in that case self-approval is permitted and shall be recorded in the activity log as a self-approval. Implement the exception explicitly; do not achieve it by weakening the general rule.

## 7. Assignment Rules

“Who may assign work to whom” is the core authorization question of this phase. It is derived from three facts about the actor and the target: their system role, their departments, and their seniority ranks.

## 7.1 Who May Assign To Whom

| Actor | May assign a task to… | Constraint |
|---|---|---|
| **ADMIN** | Any active employee, in any department. | None beyond the target being active. |
| **Department manager (Department.managerId)** | Any active employee in a department they manage, including themselves. | Target must be a member of that department. |
| **MANAGER role (without being that department's manager)** | Any active employee who reports to them, directly or indirectly, plus themselves. | Reporting chain is followed upward from the target. |
| **Employee whose level rank ≥ Lead** | Active employees in their own department whose level rank is *strictly lower* than their own, plus themselves. | Same department only. |
| **Any other employee (MEMBER)** | Themselves only. | May claim an unassigned task in their own department; may not hand work to anyone else. |

## 7.2 Requirements

| ID | Requirement | Priority |
|---|---|---|
| **TR-22** | Assignment shall be performed through a dedicated endpoint (`PATCH /v1/tasks/:id/assignee`) accepting an employee id, or null to unassign. | **MUST** |
| **TR-23** | An assignment the actor is not permitted to make shall return **403** with code `TASK_ASSIGNMENT_NOT_PERMITTED`, and the message shall state the rule that was violated — not merely "forbidden". | **MUST** |
| **TR-24** | Assigning to a deactivated employee shall be refused with 422 (`EMPLOYEE_INACTIVE`). | **MUST** |
| **TR-25** | Assigning to an employee outside the task's department shall be refused with 422 unless the actor is ADMIN. | **MUST** |
| **TR-26** | The current assignee may always release the task back to unassigned, regardless of who assigned it. Releasing a task in `IN_PROGRESS` shall also move it to `OPEN`. | **MUST** |
| **TR-27** | Reassigning an already-assigned task shall notify both the previous and the new assignee (Section 9.4). | **SHOULD** |
| **TR-28** | Deactivating an employee shall not silently drop their tasks. The deactivation endpoint shall report how many active tasks they hold, and shall accept an optional `reassignTo` parameter to move them in one operation. | **SHOULD** |

## 7.3 What the Assignee May Do

A frequent question on this kind of feature is exactly how much power the assignee has over their own task. The answer, in full:

| Action | Permitted for the assignee? |
|---|---|
| **Change status** | Yes — within the transitions of Section 6.2, except IN_REVIEW → DONE. |
| **Release the task (unassign themselves)** | Yes, always (TR-26). |
| **Reassign to a different person** | Only if their own role or rank permits it under Section 7.1. Being the assignee grants no extra assignment power. |
| **Edit title and description** | Yes. |

| Action | Permitted for the assignee? |
|---|---|
| **Change priority** | Yes, but a change to or from URGENT requires the MANAGER role or above (TR-29). |
| **Change the due date** | No. Only the creator, the department manager, or ADMIN. The assignee may request a change via a comment. |
| **Add, complete, and remove checklist items** | Yes. |
| **Add and remove tags** | Yes. |
| **Comment, and edit or delete their own comments** | Yes, subject to Section 9.1. |
| **Add or remove watchers** | Yes; may always add themselves to any task they can read. |
| **Log time against the task** | Yes, for their own time entries only. |
| **Declare a dependency on another task** | Yes, within tasks they can read. |
| **Move the task to another department** | No. Creator, department manager, or ADMIN only. |
| **Archive or delete the task** | No. Creator, department manager, or ADMIN only. |

| ID | Requirement | Priority |
|---|---|---|
| **TR-29** | Setting a task's priority to `URGENT`, or clearing it from `URGENT`, shall require the MANAGER role or above; other priority changes are open to anyone who may edit the task. | **SHOULD** |

# 8. Permission Matrix

## 8.1 Roles

| Role | Definition |
|---|---|
| **ADMIN** | Unrestricted across all departments. Manages reference data (levels, tags) and other users' accounts. |
| **MANAGER** | Holds authority over a reporting chain and, where set as Department.managerId, over a department's tasks. |
| **MEMBER** | Ordinary employee. Full authority over their own work, read access to their department. |

Roles are not seniority levels (Section 4.1). A Senior-level engineer is normally a MEMBER. Where the matrix below says "own department", membership is decided by the actor's `departmentId`, not by the reporting chain.

## 8.2 Matrix

**Y** = permitted · **O** = permitted only for records the actor owns or is assigned · **D** = permitted within the actor's own department · **—** = forbidden (403).

| Action | ADMIN | MANAGER | MEMBER |
|---|---|---|---|
| Read tasks in own department | Y | Y | Y |
| Read tasks in any department | Y | D | D |
| Create a task in own department | Y | Y | Y |
| Create a task in another department | Y | — | — |
| Edit task title / description | Y | D | O |
| Change priority (non-URGENT) | Y | D | O |

| Action | ADMIN | MANAGER | MEMBER |
|---|---|---|---|
| Set or clear URGENT priority | Y | D | — |
| Change due date | Y | D | O (creator only) |
| Assign / reassign | Y | See 7.1 | See 7.1 |
| Transition status | Y | D | O |
| Approve IN_REVIEW → DONE | Y | D | D, not own task |
| Move task between departments | Y | D | O (creator only) |
| Archive a task | Y | D | O (creator only) |
| Delete (soft) a task | Y | D | O (creator only) |
| Restore a deleted task | Y | — | — |
| Comment on a readable task | Y | Y | Y |
| Edit / delete own comment | Y | Y | Y |
| Delete another person's comment | Y | D | — |
| Log time on own assignment | Y | Y | Y |
| View another employee's time entries | Y | D | — |
| Manage seniority levels | Y | — | — |
| Manage tags | Y | Y | — |
| Set an employee's level or manager | Y | D | — |
| Deactivate an employee | Y | — | — |
| Read department task summary | Y | D | D |

### 8.3 Requirements

| ID | Requirement | Priority |
|---|---|---|
| **PR-1** | Authorization shall be enforced server-side on every endpoint. A UI that hides a control is not an implementation of a permission. | **MUST** |
| **PR-2** | An unauthenticated request shall return **401**; an authenticated request lacking permission shall return **403**. The distinction shall be correct on every endpoint (Day 9). | **MUST** |
| **PR-3** | A request for a record the actor may not read shall return **404**, not 403, so that record existence is not leaked to unauthorised callers. Where the actor may read but not modify, 403 is correct. | **SHOULD** |
| **PR-4** | Permission logic shall live in one reusable, unit-testable layer — not duplicated inline across route handlers. | **MUST** |
| **PR-5** | Every cell marked "—" in the matrix above shall have a corresponding automated test asserting a 403 or 404. | **MUST** |

## 9. Collaboration Features

### 9.1 Comments

| ID | Requirement | Priority |
|---|---|---|
| **CM-1** | Any employee who may read a task shall be able to add a comment of 1–2,000 characters. | **MUST** |

| ID | Requirement | Priority |
|---|---|---|
| **CM-2** | An author may edit their own comment. An edited comment shall expose `editedAt` so the change is visible. | **MUST** |
| **CM-3** | Comments shall be soft-deleted. A deleted comment shall render as a tombstone (“comment deleted”) rather than vanishing from the thread. | **SHOULD** |
| **CM-4** | Comments shall be listed oldest-first and shall be paginated with the same conventions as every other list. | **MUST** |
| **CM-5** | A comment body may mention an employee as `@employeeId` or an agreed equivalent. Each mentioned employee who may read the task shall receive a notification and shall be added as a watcher. | **SHOULD** |
| **CM-6** | Adding a comment shall not alter `Task.updatedAt`; a comment is not an edit of the task. | **SHOULD** |

## 9.2 Watchers

| ID | Requirement | Priority |
|---|---|---|
| **WT-1** | An employee may watch or unwatch any task they can read. | **MUST** |
| **WT-2** | The creator and the current assignee shall be watchers implicitly. A previous assignee shall remain a watcher after reassignment until they explicitly unwatch. | **SHOULD** |
| **WT-3** | The task detail response shall include the watcher count and whether the calling employee is watching. | **SHOULD** |

## 9.3 Activity Log

| ID | Requirement | Priority |
|---|---|---|
| **AL-1** | Every mutation of a task — field edit, status transition, assignment, tagging, dependency, archive, delete, restore — shall append an immutable activity record. | **MUST** |
| **AL-2** | An activity record shall capture the actor, the action, the field affected where applicable, the previous and new values, an optional reason, and a timestamp. | **MUST** |
| **AL-3** | Activity records shall never be updated or hard-deleted by application code, including when the task itself is deleted. | **MUST** |
| **AL-4** | A single request that changes several fields shall produce one record per changed field, sharing a correlation identifier so the UI can group them into one entry. | **SHOULD** |
| **AL-5** | The activity endpoint shall return newest-first and shall be paginated. | **MUST** |

## 9.4 Notifications

Notifications are in-app records only. No email, SMS, or push delivery is in scope (Section 17).

| Trigger | Recipients |
|---|---|
| Task assigned to an employee | The new assignee; the previous assignee, if any |
| Status changed | Assignee and all watchers, excluding the actor |
| Comment added | Assignee and all watchers, excluding the author |
| Mentioned in a comment | The mentioned employee |
| Priority raised to URGENT | Assignee and all watchers |
| Due date set, changed, or passed | Assignee only |

| Trigger | Recipients |
|---|---|
| Blocking task completed | Assignee of the previously blocked task |

| ID | Requirement | Priority |
|---|---|---|
| **NT-1** | An actor shall never be notified of their own action. | **MUST** |
| **NT-2** | Endpoints shall exist to list the caller's notifications (filterable by unread), to mark one read, and to mark all read. | **MUST** |
| **NT-3** | Notification creation shall not be able to fail a request. If a notification cannot be written, the underlying operation still succeeds and the failure is logged. | **SHOULD** |
| **NT-4** | Overdue detection (a due date passing with the task not DONE) shall be computed on read, not by a scheduled job — background scheduling is out of scope. | **SHOULD** |

# 10. Task Relationships & Planning

## 10.1 Subtasks

| ID | Requirement | Priority |
|---|---|---|
| **TR-30** | A task may reference a parent task via `parentTaskId`. | **SHOULD** |
| **TR-31** | Nesting shall be limited to one level: a task that already has a parent may not itself be made a parent. Attempts shall be refused with 422 (`TASK_NESTING_TOO_DEEP`). | **MUST** |
| **TR-32** | A parent and its subtasks shall belong to the same department. | **MUST** |
| **TR-33** | A parent task shall not be transitioned to `DONE` while any of its subtasks is in an active status. Refuse with 422. | **SHOULD** |
| **TR-34** | The parent's detail response shall include its subtasks' count and how many are complete. | **SHOULD** |

## 10.2 Dependencies

| ID | Requirement | Priority |
|---|---|---|
| **TR-35** | A task may declare that it is blocked by one or more other tasks. The relationship shall be directional and shall be queryable in both directions ("what blocks this" and "what does this block"). | **SHOULD** |
| **TR-36** | A task may not block itself, and the dependency graph shall remain acyclic. Both violations shall be refused with 422 (`TASK_CIRCULAR_DEPENDENCY`). | **MUST** |
| **TR-37** | A dependency on a task in another department is permitted, provided the actor may read both tasks. | **MAY** |
| **TR-38** | Completing a task shall notify the assignees of every task it was blocking (Section 9.4). | **SHOULD** |

## 10.3 Checklists, Tags, Estimates and Time

| ID | Requirement | Priority |
|---|---|---|
| **TR-39** | A task may hold an ordered list of checklist items, each with a label of 1–200 characters and a done flag. Items shall be reorderable via an explicit position field. | **SHOULD** |
| **TR-40** | Completing a checklist item shall record who completed it and when. | **SHOULD** |

| ID | Requirement | Priority |
|---|---|---|
| **TR-41** | Tags shall be system-wide reusable records with unique, case-insensitive names. Creating a task with a tag name that does not exist shall create the tag, provided the actor may manage tags; otherwise refuse with 403. | **SHOULD** |
| **TR-42** | A task shall accept at most 10 tags. Exceeding this shall be refused with 422. | **SHOULD** |
| **TR-43** | Time shall be logged as discrete entries (minutes, optional note, the date worked), never as a directly writable total. `Task.loggedMinutes` shall be derived and kept consistent with its entries. | **SHOULD** |
| **TR-44** | A time entry shall be attributable to the employee who logged it; an employee may only create and edit their own entries. | **SHOULD** |
| **TR-45** | A single time entry shall be between 1 and 1,440 minutes, and its `workedOn` date shall not be in the future. | **SHOULD** |

# 11. Listing, Filtering & Reporting

## 11.1 The Task List Endpoint

A single well-built list endpoint carries most of the product. It shall accept the following query parameters, all optional, and all combinable. Unknown parameters shall be rejected with 400 rather than silently ignored.

| Parameter | Behaviour |
|---|---|
| **status** | One or more statuses, comma-separated. Also accepts the pseudo-values `active` and `terminal`. |
| **priority** | One or more priorities, comma-separated. |
| **assigneeId** | Employee id, or the literal `me`, or `unassigned`. |
| **createdById** | Employee id, or `me`. |
| **departmentId** | Department id. Ignored — with a documented 403 — where the actor may not read that department. |
| **watchedByMe** | Boolean. Restricts to tasks the caller watches. |
| **tag** | One or more tag names, comma-separated. Multiple tags are combined with AND. |
| **search** | Case-insensitive partial match against title, description, and key. |
| **dueBefore / dueAfter** | ISO-8601 timestamps bounding the due date. |
| **overdue** | Boolean. Due date in the past and status not terminal. |
| **includeArchived** | Boolean, defaults to false. |
| **parentTaskId** | Restricts to subtasks of one parent; `none` restricts to top-level tasks only. |
| **sort** | One of `createdAt, updatedAt, dueAt, priority, status, title`, each with an optional `-` prefix for descending. Defaults to `-createdAt`. |
| **page / limit** | Pagination. Default limit 25, maximum 100. |

## 11.2 Requirements

| ID | Requirement | Priority |
|---|---|---|
| **LS-1** | Results shall be restricted to tasks the caller may read, before pagination is applied — never filtered in application code after fetching a page. | **MUST** |
| **LS-2** | Priority sorting shall order by business severity (URGENT, HIGH, NORMAL, LOW), not alphabetically. | **MUST** |
| **LS-3** | The response envelope shall carry `data`, `page`, `limit`, `total`, and `totalPages`, matching the Phase 1 convention. | **MUST** |
| **LS-4** | The list shall not issue a query per row to resolve assignee, department, or tags. Demonstrate in the pull request that the N+1 problem has been avoided (Day 14). | **MUST** |
| **LS-5** | Indexes shall exist on the columns the filters above rely on, at minimum `status`, `assigneeId`, `departmentId`, `dueAt`, and `deletedAt`. | **MUST** |
| **LS-6** | A convenience endpoint `GET /v1/tasks/mine` shall return the caller's active assigned tasks, ordered by priority then due date. | **SHOULD** |

## 11.3 Reporting Endpoints

| ID | Requirement | Priority |
|---|---|---|
| **RP-1** | A per-department summary shall return task counts grouped by status, plus overdue and unassigned counts, in one request. | **MUST** |
| **RP-2** | A per-employee workload summary shall return, for each active member of a department, their count of active tasks broken down by priority. | **SHOULD** |
| **RP-3** | Reporting endpoints shall be computed with aggregate queries in the database, not by fetching rows and counting in JavaScript. | **MUST** |
| **RP-4** | Reporting endpoints shall obey the same visibility rules as the list endpoint; a MEMBER requesting another department's summary receives 403. | **MUST** |

# 12. API Specification

All endpoints sit under `/v1` and require authentication unless noted. This table is the required surface; adding endpoints beyond it is permitted where a requirement above implies one, provided the addition is documented in the OpenAPI spec.

## 12.1 Organisation

| Method | Path | Purpose | OK |
|---|---|---|---|
| **GET** | `/seniority-levels` | List levels, ordered by rank. | **200** |
| **POST** | `/seniority-levels` | Create a level. ADMIN only. | **201** |
| **PATCH** | `/seniority-levels/:id` | Update name, rank, description, or active flag. | **200** |
| **DELETE** | `/seniority-levels/:id` | Delete a level; 409 if referenced (OR-10). | **204** |
| **PATCH** | `/employees/:id` | Update level, job title, manager, department, active flag. | **200** |
| **GET** | `/employees/:id/reports` | Direct reports of one employee. | **200** |
| **GET** | `/employees/:id/chain` | Reporting chain upward to the top. | **200** |
| **PATCH** | `/departments/:id/manager` | Set or clear the department manager. | **200** |

## 12.2 Tasks

| Method | Path | Purpose | OK |
|---|---|---|---|
| **POST** | `/tasks` | Create a task. | **201** |
| **GET** | `/tasks` | List tasks with the filters of Section 11.1. | **200** |
| **GET** | `/tasks/mine` | The caller’s active assigned tasks. | **200** |
| **GET** | `/tasks/:id` | Task detail, with nested relations (TR-14). | **200** |
| **PATCH** | `/tasks/:id` | Update title, description, priority, due date, estimate, department. | **200** |
| **DELETE** | `/tasks/:id` | Soft-delete. | **204** |
| **POST** | `/tasks/:id/restore` | Restore a soft-deleted task. ADMIN only. | **200** |
| **PATCH** | `/tasks/:id/status` | Transition status. Body: { status, reason? }. | **200** |
| **PATCH** | `/tasks/:id/assignee` | Assign, reassign, or release. Body: { assigneeId \| null }. | **200** |
| **PATCH** | `/tasks/:id/archive` | Set or clear the archived flag. | **200** |
| **GET** | `/tasks/:id/activity` | Paginated activity history, newest first. | **200** |

## 12.3 Collaboration

| Method | Path | Purpose | OK |
|---|---|---|---|
| **GET** | `/tasks/:id/comments` | List comments, oldest first, paginated. | **200** |
| **POST** | `/tasks/:id/comments` | Add a comment. | **201** |
| **PATCH** | `/comments/:id` | Edit own comment. | **200** |
| **DELETE** | `/comments/:id` | Soft-delete a comment. | **204** |
| **POST** | `/tasks/:id/watchers` | Watch a task (self, or another employee if permitted). | **201** |
| **DELETE** | `/tasks/:id/watchers/:employeeId` | Stop watching. | **204** |
| **GET** | `/notifications` | The caller’s notifications; ?unread=true supported. | **200** |
| **PATCH** | `/notifications/:id/read` | Mark one notification read. | **200** |
| **POST** | `/notifications/read-all` | Mark every notification read. | **204** |

### 12.4 Structure, Tags and Time

| Method | Path | Purpose | OK |
|---|---|---|---|
| **POST** | `/tasks/:id/checklist` | Add a checklist item. | **201** |
| **PATCH** | `/checklist-items/:id` | Update label, done flag, or position. | **200** |
| **DELETE** | `/checklist-items/:id` | Remove a checklist item. | **204** |
| **GET** | `/tags` | List tags, with usage counts. | **200** |
| **POST** | `/tags` | Create a tag. | **201** |
| **DELETE** | `/tags/:id` | Delete a tag and detach it from all tasks. ADMIN only. | **204** |
| **PUT** | `/tasks/:id/tags` | Replace the task's tag set. Body: { tags: [name] }. | **200** |
| **POST** | `/tasks/:id/dependencies` | Declare a blocking task. Body: { blockingTaskId }. | **201** |
| **DELETE** | `/tasks/:id/dependencies/:blockingTaskId` | Remove a dependency. | **204** |
| **POST** | `/tasks/:id/time-entries` | Log time. Body: { minutes, workedOn, note? }. | **201** |
| **GET** | `/tasks/:id/time-entries` | List time entries for a task. | **200** |
| **DELETE** | `/time-entries/:id` | Delete own time entry. | **204** |

### 12.5 Reporting

| Method | Path | Purpose | OK |
|---|---|---|---|
| **GET** | `/departments/:id/task-summary` | Counts by status, plus overdue and unassigned (RP-1). | **200** |
| **GET** | `/departments/:id/workload` | Per-employee active task counts by priority (RP-2). | **200** |

## 13. Validation Rules & Error Catalogue

Day 11 introduced centralized validation and error handling. Phase 2 is where that discipline is assessed: the API must fail in one consistent, predictable shape across every endpoint.

### 13.1 Error Response Shape

```
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/json

{
  "error": {
    "code": "TASK_INVALID_TRANSITION",
    "message": "A task in DONE cannot move to IN_REVIEW.",
    "details": [ { "field": "status", "issue": "from=DONE to=IN_REVIEW" } ],
    "requestId": "5f3c1a9e-..."
  }
}
```

### 13.2 Error Catalogue

| Code | HTTP | When it is returned |
|---|---|---|
| **VALIDATION_ERROR** | 400 | Body, query, or path failed schema validation. `details` lists every failing field, not just the first. |
| **UNKNOWN_QUERY_PARAM** | 400 | An unrecognised query parameter was supplied. |
| **UNAUTHENTICATED** | 401 | Missing, malformed, or expired token. |
| **FORBIDDEN** | 403 | Authenticated, but the permission matrix denies the action. |
| **TASK_ASSIGNMENT_NOT_PERMITTED** | 403 | The actor may not assign this task to that employee (TR-23). |

| Code | HTTP | When it is returned |
|---|---|---|
| **NOT_FOUND** | 404 | The resource does not exist, or the caller may not know that it does (PR-3). |
| **TASK_INVALID_TRANSITION** | 409 | The requested status transition is not in the matrix of Section 6.2. |
| **LEVEL_IN_USE** | 409 | A seniority level referenced by employees cannot be deleted. |
| **DUPLICATE_RESOURCE** | 409 | Unique constraint violated — a duplicate tag name, watcher, or dependency. |
| **EMPLOYEE_INACTIVE** | 422 | The target employee is deactivated. |
| **REASON_REQUIRED** | 422 | A transition requiring a reason was attempted without one. |
| **TASK_HAS_OPEN_BLOCKERS** | 422 | Completion attempted while a blocking dependency is incomplete. |
| **TASK_CIRCULAR_DEPENDENCY** | 422 | The dependency or reporting change would create a cycle. |
| **TASK_NESTING_TOO_DEEP** | 422 | Subtask nesting beyond one level. |
| **CROSS_DEPARTMENT_ASSIGNMENT** | 422 | Assignment outside the task's department by a non-ADMIN. |
| **LIMIT_EXCEEDED** | 422 | A bounded collection would exceed its cap — for example more than 10 tags. |
| **INTERNAL_ERROR** | 500 | Unhandled failure. The response body must never contain a stack trace. |

### 13.3 Requirements

| ID | Requirement | Priority |
|---|---|---|
| **VE-1** | Validation shall be schema-driven and shall run before any business logic or database access. | **MUST** |
| **VE-2** | Every error response shall use the shape of Section 13.1. No endpoint may return a bare string, an HTML error page, or a framework default. | **MUST** |
| **VE-3** | Validation failures shall report *all* offending fields in one response, not abort at the first. | **MUST** |
| **VE-4** | Unexpected exceptions shall be caught by one centralized error handler, logged with a request identifier, and returned as `INTERNAL_ERROR` with no internal detail leaked. | **MUST** |
| **VE-5** | Every response, success or failure, shall carry a `requestId` correlating it with the server logs (Day 13). | **SHOULD** |
| **VE-6** | String inputs shall be trimmed before validation; a field of only whitespace shall be treated as empty. | **MUST** |
| **VE-7** | Any field not listed as writable for an endpoint shall be rejected rather than silently ignored — most importantly `id`, `key`, `createdById`, `loggedMinutes`, and every timestamp. | **MUST** |

## 14. Web UI Requirements

The Phase 1 UI is extended, under the same constraint: plain HTML and vanilla JavaScript, no framework, no design work. Every screen must exercise the API rather than reimplementing its rules client-side.

| ID | Requirement | Priority |
|---|---|---|
| **UI-13** | Authentication: a login form, token storage, an authenticated app shell, and a logout control. A 401 from any call shall return the user to the login screen. | **MUST** |

| ID | Requirement | Priority |
|---|---|---|
| **UI-14** | A task list view exposing status, priority, assignee, department, search, and overdue filters, together with sorting and pagination, all driven by the query parameters of Section 11.1. | **MUST** |
| **UI-15** | A “My tasks” view backed by `GET /v1/tasks/mine`. | **MUST** |
| **UI-16** | A task detail view showing every field, the comment thread, checklist, tags, watchers, dependencies, and the activity history. | **MUST** |
| **UI-17** | A create-task form; a status control offering *only* the transitions currently legal for that task and that user; and an assignee picker listing only employees the user may assign to. | **MUST** |
| **UI-18** | The status control shall be built from a server-provided list of available transitions, not from a copy of the transition matrix in front-end code. | **SHOULD** |
| **UI-19** | A comment box with mention support, and an unread notification indicator with a list the user can mark read. | **SHOULD** |
| **UI-20** | A department summary view rendering the counts of RP-1. | **SHOULD** |
| **UI-21** | Employee administration: set an employee’s level, manager, and active flag, restricted to users whose role permits it. | **SHOULD** |
| **UI-22** | Every API error shall surface its `error.message` to the user. A 403 shall read as a permission problem, not as a generic failure. | **MUST** |
| **UI-23** | Loading states on every asynchronous action, and explicit empty states on every list. | **SHOULD** |
| **UI-24** | A simple board view grouping tasks in columns by status is permitted but not required; drag-and-drop is explicitly not required. | **MAY** |

**STILL NOT A DESIGN EXERCISE**

The same rule as Phase 1 applies: unstyled and correct beats styled and wrong. What is assessed here is whether the UI reflects the server’s rules honestly — whether the buttons a user cannot use are absent, and whether an error from the API becomes a sentence a person can read.

# 15. Non-Functional Requirements

| ID | Requirement |
|---|---|
| **NFR-1** | Node.js, Express, PostgreSQL, and Prisma, continuous with Phase 1. No change of stack. |
| **NFR-2** | Every schema change shall be a committed Prisma migration. No manual SQL applied outside migrations, and no destructive migration against existing data without a documented backfill (Section 4.3). |
| **NFR-3** | Routing, business logic, and data access shall stay separated. Permission checks and status-transition rules shall live in the service layer, callable and testable without an HTTP request. |
| **NFR-4** | Multi-write operations — a transition with its activity record and notifications, a tag replacement, a deactivation with reassignment — shall run inside a database transaction. |
| **NFR-5** | All timestamps shall be stored in UTC and serialised as ISO-8601 with an explicit offset. |
| **NFR-6** | With 5,000 tasks and 200 employees seeded, the task list endpoint shall respond within 300 ms locally at the default page size. Record the measured figure in the README. |

| ID | Requirement |
|---|---|
| **NFR-7** | Pagination shall be mandatory on every collection endpoint. No endpoint may return an unbounded list. |
| **NFR-8** | The project shall run from a clean clone via the README alone, including migrations and a seed that produces a realistic dataset: several departments, a populated hierarchy, and tasks spread across every status and priority. |
| **NFR-9** | An OpenAPI specification shall cover every endpoint, with request and response schemas and every documented error code, and shall be served as browsable docs (Day 12). |
| **NFR-10** | Feature-branch workflow with small commits, delivered as reviewed pull requests per milestone rather than one large final merge (Day 2, Day 4). |
| **NFR-11** | Automated tests (Day 10) shall cover, at minimum: every status transition including the illegal ones; every "—" cell of the permission matrix; the assignment rules of Section 7.1; cycle detection for both reporting lines and dependencies; and the list endpoint's filter combinations. |
| **NFR-12** | Structured logging (Day 13) with a request identifier on every entry. Every authorization denial shall be logged with actor, action, and target. |
| **NFR-13** | No secrets in the repository. `.env.example` shall list every required variable, including the JWT secret and its expiry. |
| **NFR-14** | The server shall survive malformed input, unknown routes, and invalid JSON without crashing the process. |

# 16. Security Requirements

| ID | Requirement |
|---|---|
| **SEC-1** | Passwords shall be bcrypt-hashed with an appropriate cost factor and shall never appear in any response, log line, or error message. |
| **SEC-2** | JWTs shall carry an expiry. An expired token shall produce 401, never a silent extension. |
| **SEC-3** | The authenticated principal shall be derived from the verified token only. A client-supplied `employeeId` in a body or query shall never be trusted to establish identity. |
| **SEC-4** | Every endpoint shall be authenticated by default; exemptions shall be explicit and listed in the README. |
| **SEC-5** | Deactivated employees shall be denied login, and any existing token they hold shall stop working at the next request. |
| **SEC-6** | All database access shall go through Prisma's parameterised queries. Any raw SQL shall be parameterised and justified in `docs/decisions.md`. |
| **SEC-7** | Comment and description bodies shall be stored as plain text and escaped on output; the UI shall never inject a stored string as HTML. |
| **SEC-8** | The login endpoint shall be rate-limited, and a failed login shall not reveal whether the account exists. |

# 17. Out of Scope — Phase 2

Excluded deliberately. Building any of these does not earn credit and will be raised in review as scope that was not requested:

- **File uploads and attachments.** External URL references only.

- **Email, SMS, or push notification delivery.** In-app notification records only.
- **Background jobs, schedulers, and cron.** Everything is computed on request (NT-4).
- **Real-time updates** — websockets, server-sent events, polling. The UI refreshes on user action.
- **Custom fields, custom statuses, or per-department workflow configuration.** One fixed workflow system-wide.
- **Multiple assignees, teams as first-class entities, or cross-department projects.**
- **Rich text, markdown rendering, or an editor.** Plain text throughout.
- **Full-text search infrastructure.** A database `ILIKE` is sufficient at this scale.
- **Deployment, containerisation, or CI pipelines.** Local execution only.
- **Multi-tenancy.** One organisation, one database.

# 18. Stretch Goals

> **OPTIONAL — DOES NOT AFFECT ACCEPTANCE**
> Attempt these only once every MUST in Sections 4–16 is complete and merged. A partial stretch goal in an otherwise incomplete submission counts against the delivery, not for it.

- **Saved views.** Let a user persist a named filter combination and recall it by name.
- **Bulk operations.** One endpoint that applies a status change, an assignment, or a tag to many task ids at once, atomically, with a per-id result report.
- **Recurring tasks.** On completing a task marked recurring, create its next instance from a stored interval — computed at completion time, without a scheduler.
- **Task templates.** Create a task, with its checklist pre-populated, from a stored template.
- **CSV export** of a filtered task list, honouring the same visibility rules as the list endpoint.
- **Cursor pagination** on the activity log, alongside the page-based scheme used elsewhere.
- **Optimistic concurrency.** Reject an update whose `If-Unmodified-Since` or version field is stale, with 412, so two simultaneous editors cannot silently overwrite each other.

# 19. Deliverables & Acceptance Criteria

## 19.1 Deliverables

- [ ] The extended StaffDesk repository, with Phase 1 functionality intact (Section 2.1).
- [ ] Prisma migrations for every schema change, including the staged backfill for `Employee.levelId`.
- [ ] A seed script producing the dataset described in NFR-8.
- [ ] An automated test suite meeting NFR-11, runnable with a single documented command.
- [ ] An OpenAPI specification covering the full surface, with browsable docs (NFR-9).
- [ ] An updated README: setup, the endpoint list, the permission model in prose, and the NFR-6 measurement.
- [ ] `docs/decisions.md`, recording every delegated decision and every deviation from Section 2's assumptions.
- [ ] One pull request per milestone, each independently reviewable, merged into `main`.

- A short written walkthrough demonstrating: a task moving through its full lifecycle; a rejected illegal transition; and a rejected unauthorised assignment.

### 19.2 Acceptance Criteria

| Ref. | Acceptance Check |
|---|---|
| **Section 4** | Levels are reference data with working CRUD; every employee has a level; reporting lines exist and cycles are refused; the migration ran against existing rows without data loss. |
| **Sections 5–6** | Tasks carry every specified field; keys are unique and immutable; every legal transition works and every illegal one returns 409 with the right code; reasons are captured; completedAt is set and cleared correctly. |
| **Sections 7–8** | Each row of the assignment table and each cell of the permission matrix behaves as specified, proven by tests; 401 and 403 are never confused; permission logic exists in one place. |
| **Sections 9–10** | Comments, watchers, notifications, checklists, tags, dependencies, and time entries work; the activity log reconstructs a task's full history; cycles and nesting limits are enforced. |
| **Section 11** | Every filter works alone and in combination; visibility is applied before pagination; priority sorts by severity; no N+1 in the list path; summaries are computed by aggregate queries. |
| **Sections 13–16** | One error shape everywhere with the catalogue's codes; validation reports all fields at once; OpenAPI matches the implementation; tests pass from a clean clone; no secrets committed. |
| **Section 14** | The UI performs the full lifecycle against the real API, offers only legal actions, and surfaces API errors as readable messages. |

## 20. Milestones & Timeline

Four milestones, each ending in its own pull request. The estimate assumes full-time work; agree actual dates with your mentor and report slippage on the day it becomes visible, not at the milestone boundary.

| # | Milestone | Contents | Estimate |
|---|---|---|---|
| **M1** | **Organisation** | Section 4 in full: levels, migration and backfill, reporting lines, department managers, employee filters. | ≈ **3 days** |
| **M2** | **Task core** | Sections 5, 6, 7, 8: the task model, the status machine, assignment rules, the permission layer, and the activity log. | ≈ **5 days** |
| **M3** | **Collaboration & query** | Sections 9, 10, 11: comments, watchers, notifications, checklists, tags, dependencies, time, the list endpoint, and reporting. | ≈ **4 days** |
| **M4** | **Hardening & interface** | Sections 13, 14, 15, 16: the error catalogue, tests, OpenAPI, logging, and the UI extension. | ≈ **4 days** |

Raise questions against requirement IDs as you hit them rather than saving them for the end. A specification this size always contains something ambiguous or wrong; finding it and asking is part of the work, not an interruption to it.

# Appendix A — Enumeration Reference

### *TaskStatus*

```
OPEN | IN_PROGRESS | BLOCKED | IN_REVIEW | DONE | CANCELLED
```

Active statuses: OPEN, IN_PROGRESS, BLOCKED, IN_REVIEW. Terminal statuses: DONE, CANCELLED.

### *TaskPriority*

```
URGENT | HIGH | NORMAL | LOW   (severity order, used by LS-2)
```

### *SystemRole*

```
ADMIN | MANAGER | MEMBER
```

### *ActivityAction*

```
CREATED | STATUS_CHANGED | ASSIGNED | UNASSIGNED | FIELD_UPDATED | TAG_ADDED |
TAG_REMOVED | DEPENDENCY_ADDED | DEPENDENCY_REMOVED | CHECKLIST_ITEM_ADDED |
CHECKLIST_ITEM_COMPLETED | COMMENTED | TIME_LOGGED | ARCHIVED | DELETED | RESTORED
```

### *NotificationType*

```
TASK_ASSIGNED | TASK_UNASSIGNED | STATUS_CHANGED | COMMENT_ADDED | MENTIONED |
PRIORITY_RAISED | DUE_SOON | OVERDUE | BLOCKER_CLEARED
```

# Appendix B — Sample Payloads

Illustrative only. Exact field naming is yours to settle, provided it is consistent across the API and documented in the OpenAPI spec.

### *Create a task*

```
POST /v1/tasks

{
  "title": "Add pagination to the employee export",
  "description": "The export times out past ~2k rows.",
  "departmentId": 3,
  "assigneeId": 41,
  "priority": "HIGH",
  "dueAt": "2026-09-18T15:00:00+03:00",
  "estimateMinutes": 240,
  "tags": ["performance", "reporting"]
}
```

### *Transition a status*

```
PATCH /v1/tasks/108/status

{ "status": "BLOCKED", "reason": "Waiting on the schema change in TSK-1039." }
```

### *Task detail response (abridged)*

```
{
  "id": 108, "key": "TSK-1042",
  "title": "Add pagination to the employee export",
  "status": "IN_PROGRESS", "priority": "HIGH",
  "assignee": { "id": 41, "fullName": "Nour Hassan", "level": "Senior" },
  "createdBy": { "id": 12, "fullName": "Mazen Adeeb" },
  "department": { "id": 3, "name": "Engineering" },
  "tags": ["performance", "reporting"],
  "dueAt": "2026-09-18T15:00:00+03:00", "isOverdue": false,
  "checklist": { "total": 3, "done": 1 },
  "blockedBy": [ { "id": 105, "key": "TSK-1039", "status": "IN_REVIEW" } ],
  "watcherCount": 4, "isWatching": true,
  "availableTransitions": ["IN_REVIEW", "BLOCKED", "OPEN"],
  "createdAt": "2026-09-02T09:14:22+03:00", "completedAt": null
}
```


# Appendix C — Glossary

**Actor**
The authenticated employee making a request. Always derived from the token, never from the request body.

**Activity record**
One immutable entry describing a single change to a task. Together they form the task's history.

**Assignee**
The one employee currently responsible for a task. A task has at most one.

**Blocking dependency**
A task that must be completed before another may be completed.

**Reporting chain**
The sequence of managers above an employee, followed upward until an employee with no manager.

**Seniority level**
Ordered reference data describing rank. Distinct from job title and from system role (Section 4.1).

**Soft delete**
Marking a row deleted rather than removing it, so history and references survive.

**Terminal status**
A status from which no further work is expected — DONE or CANCELLED.

**Transition**
A permitted move from one task status to another, as defined by Section 6.2.

**Watcher**
An employee who receives notifications about a task without being responsible for it.

# Appendix D — Reference Systems Surveyed

This specification was drafted after reviewing how established task systems model the same problem. The column on the right records what StaffDesk borrowed and, more usefully, what it deliberately left behind. If you want to see any of these ideas at full scale, these are the products to look at.

| System | What it does | What StaffDesk takes from it |
|---|---|---|
| **ClickUp** | Statuses grouped into Not Started / Active / Done categories; priority as a four-level flag; checklists and watchers on every task. | Borrowed: the status *categories* of Section 6.1 and the four-level priority. Dropped: custom status sets, custom fields, and the space/folder/list hierarchy. |
| **Jira** | Immutable issue keys (PROJ-123), an explicit workflow with permitted transitions, a full changelog, and blocking issue links. | Borrowed: the reference key (TR-2), the transition matrix, the activity log, and dependencies. Dropped: configurable workflows, screens, and sprints. |
| **Asana** | Assignee-and-followers split; due dates; subtasks; task-level comment threads. | Borrowed: the single-assignee-plus-watchers model of Section 9.2 and one-level subtasks. Dropped: multi-project membership. |
| **Linear** | A small fixed status set, a strong opinion about defaults, and a fast filtered list as the primary surface. | Borrowed: one fixed workflow system-wide, and the emphasis on a single well-built list endpoint (Section 11). |
| **Trello** | Board columns mapped to status; drag-and-drop as the main interaction. | Borrowed: status-as-column, offered as an optional view (UI-24). Dropped: drag-and-drop, explicitly not required. |

Product names are the trademarks of their respective owners and are referenced here only to describe well-known design patterns.